

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

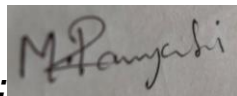
5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

*I **M.Ramya Sri(192372144)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. XML Parsing Algorithm

Problem: Parse an XML document containing structured user/product information, traverse its hierarchical nodes, extract required data, and display it.

Pseudocode

ALGORITHM XML_Parsing

BEGIN

INPUT XML file name

Step 1: Load the XML document

Open the XML file

IF file cannot be opened THEN

Display "Error: XML file not found"

STOP

END IF

Step 2: Parse the XML document

Create an XML Document Object

Parse the XML file

IF XML format is invalid THEN

Display "Error: Invalid XML document"

STOP

END IF

Step 3: Access the root element

Get the root element of the XML document

Display "Root Element: " + root element name

Step 4: Traverse child nodes

FOR each child node of the root element DO

IF child node is an element THEN

Get the node name

Display node name

Step 5: Access sub-elements

FOR each sub-node of the current node DO

IF sub-node is an element THEN

Get sub-node name

Get sub-node value

```
        Display:
            sub-node name
            sub-node value
        END IF
```

```
    END FOR
END IF
```

```
END FOR
```

Step 6: Display extracted information
Format the retrieved values in a readable manner
Display all required user/product details

Step 7: Close the XML document

END

Example Output

Root Element: Products

Product 1

ID : P101

Name : Laptop

Price : 55000

Product 2

ID : P102

Name : Smartphone

Price : 25000

Logic: The algorithm first loads and validates the XML document, identifies the root, then uses nested traversal to access hierarchical elements and extract their values.

2. MVC Workflow

Problem: Represent the complete flow of a user request through the Controller, Model, database, and View.

Pseudocode

ALGORITHM MVC_Workflow

BEGIN

Step 1: Receive user request
User sends a request from the browser
Send request to the web application

Step 2: Controller receives request

Controller identifies the requested operation
Read request parameters

Step 3: Validate request

IF request data is invalid THEN
 Prepare an error response
 Send response to View
ELSE
 Continue processing
END IF

Step 4: Call Model

Controller sends required data to Model

Step 5: Model processes request

Model applies application/business logic
Establish connection with database

Step 6: Database interaction

Execute required SQL operation
Retrieve or update required data
Close database connection

Step 7: Return result

Model sends processed data back to Controller

Step 8: Select View

Controller selects the appropriate View
Pass processed data to the View

Step 9: Generate response

View uses the received data
Generate dynamic HTML response

Step 10: Send response

Controller sends the generated response to the browser

Step 11: Display result

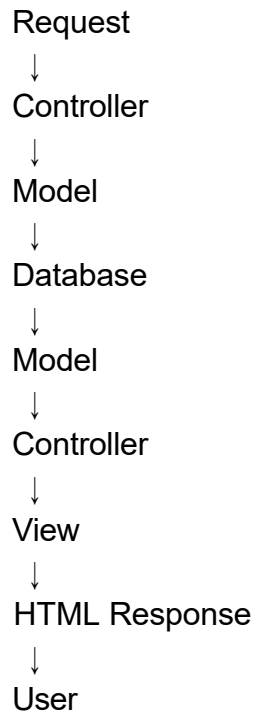
Browser displays the response to the user

END

MVC Flow

User





Logic: The Controller manages the request, the Model handles business logic and database operations, and the View presents the processed data to the user.

3. JSP Request Flow

Problem: Explain the request–response cycle of a JSP-based web application, including backend processing and dynamic content generation.

Pseudocode

ALGORITHM JSP_Request_Flow

BEGIN

Step 1: Client initiates request

 User enters JSP URL in the browser

 Browser sends HTTP request to the web server

Step 2: Server receives request

 Web server receives the request

 Identify the requested JSP page

Step 3: Check JSP status

 IF JSP page is requested for the first time

 OR JSP page has been modified THEN

 Translate JSP into Servlet source code

 Compile the Servlet

END IF

Step 4: Execute JSP Servlet

Create request and response objects

Execute the generated Servlet

Step 5: Process backend request

Read input parameters from the request

IF backend processing is required THEN

Call Servlet or JavaBean

Connect to database if required

Execute required database operation

Retrieve required data

END IF

Step 6: Generate dynamic content

Pass retrieved data to JSP

JSP combines HTML with dynamic data

Generate final HTML page

Step 7: Create HTTP response

Server places generated HTML into HTTP response

Step 8: Send response

Send HTTP response to the client browser

Step 9: Display page

Browser receives HTML

Render the page

Display dynamic content to the user

END

JSP Request Flow

Client Browser



HTTP Request



Web Server



JSP Page



JSP → Servlet Translation



Servlet Execution



Backend / Database



Dynamic Data



JSP Response Generation



HTML Response



Browser

Important: JSP pages are translated into Servlets by the JSP container and then executed to generate the dynamic response.

4. Database Retrieval

Problem: Retrieve records from a database using a sequence of connection, query execution, result processing, and display operations.

Pseudocode

ALGORITHM Database_Retrieval

BEGIN

Step 1: Initialize database details

Set database URL

Set username

Set password

Set required SQL query

Step 2: Load database driver

Load the appropriate database driver

Step 3: Establish connection

Connect to the database

IF connection fails THEN

Display "Database connection failed"

STOP

END IF

Step 4: Create statement

Create a database statement object

Step 5: Execute SQL query

Execute SELECT query

Store the returned records in ResultSet

Step 6: Check records

```
IF ResultSet contains no records THEN
    Display "No records found"
ELSE
```

```
    Step 7: Process records
    WHILE another record exists DO
```

```
        Read required fields
        Store field values
```

```
        Display the retrieved values
```

```
    END WHILE
```

```
END IF
```

```
Step 8: Close resources
Close ResultSet
Close Statement
Close database connection
```

```
Step 9: End operation
Display "Data retrieval completed"
```

```
END
```

Example

SQL Query:

```
SELECT id, name, email FROM users;
```

Output:

ID	Name	Email
101	Arun	arun@gmail.com
102	Priya	priya@gmail.com
103	Ravi	ravi@gmail.com

Logic: The algorithm ensures that the database connection is established before the query is executed and that resources are closed after processing the result.

5. PHP Validation

Problem: Validate user input submitted through a web form using PHP concepts. Empty fields, email format, and password requirements must be checked before accepting the form.

Pseudocode

ALGORITHM PHP_Form_Validation

BEGIN

Step 1: Receive form data

Read name

Read email

Read password

Read confirm password

Step 2: Remove unnecessary spaces

Trim name

Trim email

Trim password

Trim confirm password

Set error_count = 0

Step 3: Validate name

IF name is empty THEN

Display "Name is required"

error_count = error_count + 1

END IF

Step 4: Validate email

IF email is empty THEN

Display "Email is required"

error_count = error_count + 1

ELSE IF email format is invalid THEN

Display "Enter a valid email address"

error_count = error_count + 1

END IF

Step 5: Validate password

IF password is empty THEN

Display "Password is required"

error_count = error_count + 1

ELSE IF password length is less than 8 THEN

Display "Password must contain at least 8 characters"

error_count = error_count + 1

END IF

Step 6: Validate confirm password

IF confirm password is empty THEN

Display "Confirm password is required"

error_count = error_count + 1

```
ELSE IF password is not equal to confirm password THEN
    Display "Passwords do not match"
    error_count = error_count + 1
END IF
```

Step 7: Check validation result

```
IF error_count = 0 THEN

    Display "Validation successful"
    Allow form submission

    Process the valid input
    Store data or continue to the next page

ELSE

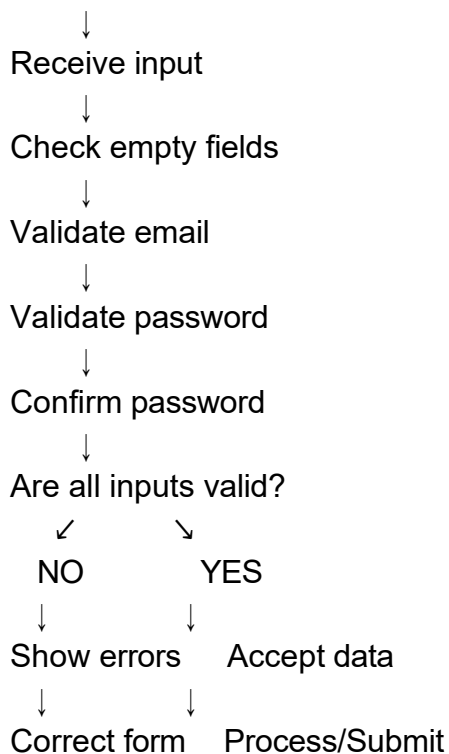
    Display "Please correct the errors"
    Prevent form submission

END IF
```

END

Validation Flow

User enters form data



Overall Algorithm Design Summary

Question	Main Concept	Important Control Structures
1. XML Parsing	XML document and node traversal	FOR, IF
2. MVC Workflow	Controller–Model–View interaction	IF, sequential flow
3. JSP Request Flow	JSP request and response cycle	IF, processing steps
4. Database Retrieval	SQL data retrieval	IF, WHILE
5. PHP Validation	Server-side form validation	IF, validation conditions